

MBQ: Modality-Balanced Quantization for Large Vision-Language Models

26.06.12

Vision-Language Model vs. Large Language Model

- 언어 모델과 달리, VLM과 같은 경우 language 토큰과 vision 토큰 간 sensitivity(gradient)의 차이가 존재
- 기존 PTQ 방식은 시각 토큰과 언어 토큰을 동일하게 취급
 - ⇒ 다른 모달리티의 토큰을 동일하게 다루면 insensitive한 모달리티를 over-emphasize 한다.

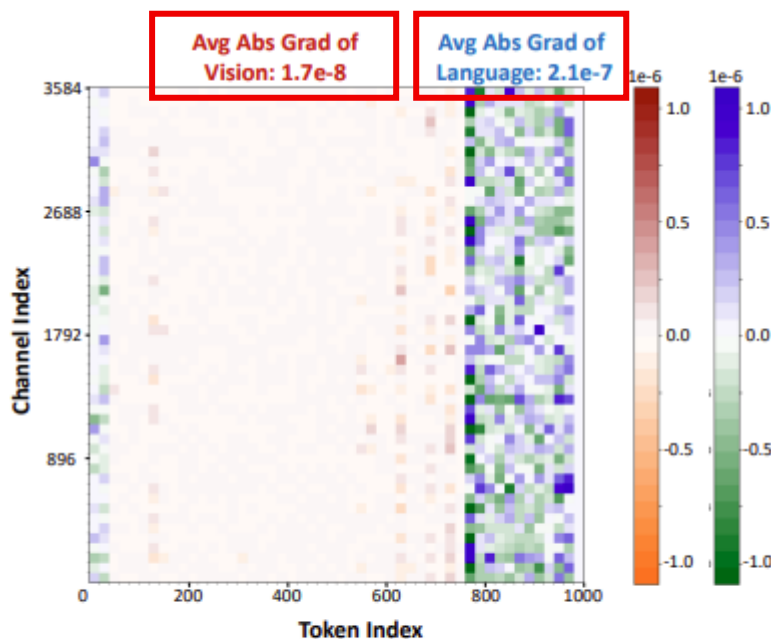


Figure 1. The gradients of loss function w.r.t. the token features of the 13th transformer block in the LLaVA-onevision-7B VLM on COCO caption dataset [8, 10]. The red and orange represent vision tokens, and the blue and green represent language tokens.

위 그림과 같이 language 토큰(파랑)의 gradient 값이 vision 토큰(주황)보다 10x 이상 크다. 이는 language 토큰이 loss function을 vision 토큰보다 10배 이상 영향을 주는 것을 의미한다.

■ Over-emphasize insensitive vision token

Language token과 vision token을 동일하게 취급하면, 상대적으로 덜 중요한 vision token에 대한 양자화 오차를 줄이는데 과도하게 낭비

기존 방식들은 모달리티의 구분 없이 토큰을 동일하게 취급하여 Reconstruction Loss를 최소화

이 경우 양자화 오차가 좀 많이 발생해도 최종 답변 성능에 아무런 지장이 없는 vision 토큰의 오차를 조금이라도 더 줄이기 위해 불필요하게 집중

⇒ 이로 인해 성능에 더 직접적인 영향을 미치는 sensitive한 language token을 더 정밀하게 양자화할 scale factor에 대한 여력을 빼앗김

■ Modality-Balanced Quantization

모달리티 별 토큰의 민감도를 고려해서 reconstruction loss를 구성하자

■ Channel-Wise Equalization

2.2. Channel-Wise Equalization

Existing PTQ methods automatically search for optimal hyperparameters of quantization by minimizing the reconstruction error of each transformer block during a calibration process. A series of popular methods [27, 43] aim to equalize outliers in weight and activation matrices by channel-wise equalization (CWE). Specifically, they search the CWE factors $\mathbf{E}$ by minimizing the Mean Square Error (MSE) loss in each transformer block. Taking weight-only quantization as an example, the objective of CWE is shown in the following equation:

$$\mathbf{E}^* = \underset{\mathbf{E}}{\operatorname{argmin}} ||Q(\mathbf{W} * \mathbf{E})(\mathbf{E}^{-1} * \mathbf{X}) - \mathbf{W}\mathbf{X}||^2, \quad (5)$$

where Q means the quantization function.

SmoothQuant 방식처럼 Channel-wise Equalization Factor를 통해, activation의 quantization 난이도를 weight로 넘긴다 => $\mathbf{E}$

즉, 가장 recon error가 작아지는 $\mathbf{E}$ 를 찾자

하지만, 기존 CWE는 모달리티의 차이를 고려 X

시각 데이터는 일반적으로 높은 중복성을 지니며 작은 변화에 관대하다 => gradient가 10배 차이 나는 원인

To demonstrate the impact of accounting for modality-specific sensitivities during calibration, we conduct an oracle experiment by applying a modality-balancing factor of 0.1 to the vision tokens' reconstruction loss during CWE calibration. The optimization objective, referred to as balanced CWE, is shown below:

$$\mathbf{E}^* = \underset{\mathbf{E}}{\operatorname{argmin}} [||Q(\mathbf{W} * \mathbf{E})(\mathbf{E}^{-1} * \mathbf{X}_l) - \mathbf{W}\mathbf{X}_l||^2] \quad (6)$$

$$+ 0.1 ||Q(\mathbf{W} * \mathbf{E})(\mathbf{E}^{-1} * \mathbf{X}_v) - \mathbf{W}\mathbf{X}_v||^2], \quad (7)$$

where the $\mathbf{X}_l$ and $\mathbf{X}_v$ mean the language and vision tokens.

The results of our ablation study are shown in Tab. 1, with a heuristic selected modality-balancing factor, the balanced CWE easily surpasses the performance of CWE by

Model	FP16	W3		
		RTN	CWE	Balanced CWE
LLaVA-ov-7B	46.00	34.67	36.56	40.22
InternVL2-8B	48.00	43.67	44.78	46.33

Table 1. The accuracy results (%) on MMMU benchmark after W3 quantization. The calibration dataset is selected from COCO caption dataset [10]. The LLaVA-ov-7B is short for LLaVA-onevision-7B.

위와 같이 balanced CWE 즉, 임의의 가중치 차이를 줘서 진행했더니 결과가 좋더라

그럼 automatic modality-balancing approach 즉, 데이터를 바탕으로 이 balanced 가중치를 정하면 더 좋지 않을까? => MBQ

3.2. Modality-Balanced Quantization (MBQ)

Given that the sensitivity differences between vision and language tokens may vary across layers and VLM families, exploring an automatic modality-balancing approach could further enhance the performance of the quantized model.

In this section, we aim to derive an approach for allocating the optimal Modality-Balanced factors to each layer by directly minimizing the change in the SFT loss function. Specifically, we employ the first-order Taylor approximation in Eq. (8) to determine how the SFT loss L changes in response to a small perturbation Δ in output activation $\mathbf{Y}$ of each linear layer:

$$L(\mathbf{Y} + \Delta) \simeq L(\mathbf{Y}) + \mathbf{g}^T * \Delta, \quad (8)$$

where $\mathbf{g}^T$ represents the gradient of the output activation $\mathbf{Y}$.

The change in SFT loss caused by quantization can be expressed as the following equation:

$$||L(\hat{\mathbf{Y}})|| \simeq ||\mathbf{g}^T * \Delta|| \quad (9)$$

$$= ||\mathbf{g}_v^T * \Delta_v + \mathbf{g}_l^T * \Delta_l|| \quad (10)$$

$$\leq ||\mathbf{g}_v^T * \Delta_v|| + ||\mathbf{g}_l^T * \Delta_l|| \quad (11)$$

$$\leq |\bar{\mathbf{g}}_v| * ||\Delta_v|| + |\bar{\mathbf{g}}_l| * ||\Delta_l|| \quad (12)$$

$$= |\bar{\mathbf{g}}_v| * ||\mathbf{Y}_v - \hat{\mathbf{Y}}_v|| + |\bar{\mathbf{g}}_l| * ||\mathbf{Y}_l - \hat{\mathbf{Y}}_l||, \quad (13)$$

where $\mathbf{Y}_v$ and $\hat{\mathbf{Y}}_v$ represent the output vision tokens of the FP16 linear layer and the quantized linear, respectively. $\mathbf{Y}_l$ and $\hat{\mathbf{Y}}_l$ represent the output language tokens of the FP16 linear layer and the quantized linear, respectively. The $|\bar{\mathbf{g}}_v|$ and $|\bar{\mathbf{g}}_l|$ represent the average absolute gradient of each linear layer's output vision and language tokens, respectively.

In this paper, we combine MBQ with channel-wise equalization to search optimal equalization factors for better performance.

0.1 과 같이 임의로 값을 정하는 게 아닌, gradient G만큼을 곱해서 reconstruction loss에 반영하자

MBQ를 활용하여 optimal Channel-Wise Equalization factor를 구하자

$$\min_{\mathbf{E}} [|\bar{\mathbf{g}}_v| * \|\mathbf{W}\mathbf{X}_v - Q(\mathbf{W} * \mathbf{E})Q(\mathbf{E}^{-1} * \mathbf{X}_v)\|] \quad (14)$$

$$+ |\bar{\mathbf{g}}_l| * \|\mathbf{W}\mathbf{X}_l - Q(\mathbf{W} * \mathbf{E})Q(\mathbf{E}^{-1} * \mathbf{X}_l)\|], \quad (15)$$

where $\mathbf{X}_v$ and $\mathbf{X}_l$ represent the input token and language activations of each linear layer, respectively.

Secondly, to accelerate the decode stage of VLMs, we only quantize weights to reduce the memory overhead, aiming to minimize the balanced reconstruction error by the following objective:

$$\min_{\mathbf{E}} [|\bar{\mathbf{g}}_v| * \|\mathbf{W}\mathbf{X}_v - Q(\mathbf{W} * \mathbf{E})(\mathbf{E}^{-1} * \mathbf{X}_v)\|] \quad (16)$$

$$+ |\bar{\mathbf{g}}_l| * \|\mathbf{W}\mathbf{X}_l - Q(\mathbf{W} * \mathbf{E})(\mathbf{E}^{-1} * \mathbf{X}_l)\|], \quad (17)$$

Notably, unlike directly using the heuristically selected MSE-based balanced CWE loss in Sec. 3.1, our derived reconstruction error loss function relies on **Mean Absolute Error (MAE)**. Our ablation study in Sec. 4.3 demonstrates that minimizing MAE-based reconstruction loss in MBQ yields better results than using an MSE-based one.

위 수식처럼 각각의 vision 토큰과 language 토큰을 분리하여 해당 gradient 만큼의 가중치를 포함하여 최적 CWE E를 찾자

즉, 언어 토큰의 gradient가 더 크기 때문에, 언어 토큰의 양자화 오차를 더 크게 반영하는 Channel-Wise Equalization factor를 찾자

■ Weight-Activation Quantization

Prefill 단계에서는 Weight-Activation quantization 진행

⇒ 입력 이미지와 쿼리에 대한 인코딩 단계(이미지 분석 및 이해)로 인해 Matrix-Matrix 연산에 따른 연산 속도 개선 필요

■ Weight-Only Quantization

Decode 단계에서는 Weight-Only Quantization 진행

- ⇒ 이후 VQA와 같이 답변을 생성하는 과정은 한 토큰씩 생성하며 연산 속도보다는 메모리 대역폭이 중요
- ⇒ 매번 Activation에 대한 scale을 runtime 시 구하는 동적 양자화(Dynamic Quantization) 필요. 따라서, Prefill 단계처럼 입력 토큰이 많을 때는 연산 속도의 증가가 더 큰 파이를 차지하지만, Decode 단계에서는 매번 단 1개의 토큰 (1D vector)만 들어옴. 연산해야 할 데이터 크기는 아주 미미한데 매번 scale factor를 구하고 데이터를 변환하는 오버헤드가 더 커져 배보다 배꼽이 커지는 상황

💡 한눈에 보는 요약 표

구분	프리필 (Prefill) 단계	디코드 (Decode) 단계
목적	입력(텍스트+이미지) 분석 및 이해 1	답변 토큰 한 글자씩 순차 생성 1 3
입력 활성화 값	수백~수천 개의 토큰 (거대한 2D 행렬) 1	단 1개의 토큰 (얇은 1D 벡터) 3
주요 연산	GEMM (Matrix-Matrix 곱셈) 1	GEMV (Matrix-Vector 곱셈) 3
하드웨어 병목	Compute-bound (연산 속도가 중요) 1 2	Memory-bound (메모리 대역폭이 중요) 3 4
KV Cache	신규 생성 및 저장 1	기존 캐시 읽기 및 누적 활용 3

Model	Stage	FP16 (ms)	W3A16 (ms)	W4A8 (ms)
ViT	Prefill (729 tokens)	11.2	-	9.7
	Prefill (512 tokens)	68.8	-	59.4
VLM	Prefill (1024 tokens)	109.0	-	92.7
	Decode	29.6	21.1	26.3

Table 9. The end-to-end speed up of LLaVA-onevision-7B on RTX4090 with fused GPU kernels.

Weight-Only가 실제로 더 빠른 것을 알 수 있음

■ Experiments

Task: Image Captioning

Model: LLaVA-onevision, InternVL2, Qwen2-VL

Baseline PTQ

[Weight-Only]: RTN, AWQ, GPTQ

[Weight-Activation]: RTN, SmoothQuant

■ Ablation: MAE vs. MSE

BitWidth	Method	Components				MMMU ($\uparrow$)	SEED ($\uparrow$)
		Channel-wise Equalization	COCO Calib.	Modality-Balance (MSE)	Modality-Balance (MAE)		
FP16	-	-	-	-	-	46.0	71.1
W3A16	RTN	$\times$	$\times$	$\times$	$\times$	34.7	10.4
	AWQ	$\checkmark$	$\times$	$\times$	$\times$	36.6	51.5
	-	$\checkmark$	$\checkmark$	$\times$	$\times$	38.7	61.8
	-	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	40.8	64.8
	MBQ	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	42.0	66.4
W4A8	RTN	$\times$	$\times$	$\times$	$\times$	38.2	48.4
	SQ	$\checkmark$	$\times$	$\times$	$\times$	30.9	41.6
	-	$\checkmark$	$\checkmark$	$\times$	$\times$	29.2	10.2
	-	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	41.9	63.5
	MBQ	$\checkmark$	$\checkmark$	$\times$	$\checkmark$	42.6	64.4

Table 4. The ablation study on LLaVA-onevision-7B with W3A16 and W4A8 quantization. “SQ” is short for SmoothQuant.

■ Ablation: Combination with Rotation-based Quantization

Model	Bitwidth	Method	MMMU	OCRBench	VizWiz
LLaVA-one vision-72B	FP16	-	56.1	73.2	69.2
	W4A8	QuaRot	53.0	70.8	67.9
		SpinQuant	52.6	70.9	61.0
		MBQ (Rot)	55.8	72.5	69.0
InternVL2 -26B	FP16	-	47.1	77.9	66.2
	W4A8	QuaRot	44.2	76.7	62.1
		SpinQuant	43.7	76.8	64.5
		MBQ (Rot)	47.4	77.8	65.6
Qwen-VL -72B	FP16	-	61.1	79.9	76.0
	W4A8	QuaRot	49.7	78.1	71.4
		SpinQuant	50.8	77.4	69.3
		MBQ (Rot)	60.0	79.9	75.4

Table 5. Rotation-based quantization results on the large models of LLaVA-onevision, InternVL2, and Qwen2-VL families.

Model	Bitwidth	Method	MMMU	SEED	OCRBench	VizWiz	ScienceQA	TextVQA	Average (↑)
LLaVA-onevision-7B	FP16	-	46.0	74.9	62.2	60.4	85.4	76.1	67.5
	W3A16	RTN	34.7	5.9	35.9	59.2	86.2	60.9	47.1
		GPTQ	41.9	72.9	55.7	56.4	86.4	71.3	64.1
		AWQ	36.6	53.0	59.3	58.5	83.2	73.0	60.6
		MBQ	42.0	69.7	61.1	60.7	85.0	73.3	65.3
	W4A8	RTN	38.2	50.3	40.1	58.2	88.3	61.5	56.1
		SQ	30.9	42.7	32.0	56.7	87.1	56.9	51.1
		MBQ	42.6	67.7	52.3	58.9	88.5	68.3	63.1
InternVL2-8B	FP16	-	48.0	76.0	76.5	61.1	96.2	77.0	72.5
	W3A16	RTN	43.7	74.9	74.0	56.0	95.6	74.6	69.8
		GPTQ	41.7	73.4	70.2	59.9	89.5	73.1	68.0
		AWQ	44.8	75.2	74.7	58.9	95.5	74.2	70.6
		MBQ	46.9	75.4	75.1	58.7	95.6	75.1	71.1
	W4A8	RTN	44.3	74.0	72.0	57.1	95.5	73.1	69.3
		SQ	41.8	73.8	70.9	54.2	95.1	72.6	68.1
		MBQ	45.6	74.3	73.0	56.5	95.8	72.3	69.6
Qwen2-VL-7B	FP16	-	50.6	76.4	80.7	68.3	85.1	82.0	73.8
	W3A16	RTN	44.9	74.8	60.0	65.2	81.5	71.2	66.3
		GPTQ	43.1	73.7	74.8	64.3	79.7	76.7	68.7
		AWQ	44.7	75.1	76.9	68.0	82.5	79.5	71.1
		MBQ	47.9	74.8	76.8	67.7	82.8	79.9	71.6
	W4A8	RTN	43.8	74.9	60.3	58.9	78.9	71.0	64.6
		SQ	45.9	75.0	57.1	52.3	80.9	68.2	63.2
		MBQ	47.2	75.1	72.8	59.3	81.2	75.0	68.4

Table 2. Main results on the small models of LLaVA-onevision, InternVL2, and Qwen2-VL families. “SQ” is short for SmoothQuant.

Model	Bitwidth	Method	MMMU	SEED	OCRBench	VizWiz	ScienceQA	TextVQA	Average (↑)
LLaVA-onevision-72B	FP16	-	56.1	78.1	73.2	69.2	90.0	79.3	74.3
	W3A16	RTN	53.9	77.4	68.2	66.1	89.5	77.4	72.1
		GPTQ	52.7	76.0	69.7	68.3	89.3	77.9	72.3
		AWQ	33.4	71.2	48.7	49.3	69.2	58.8	55.1
		MBQ	54.4	77.6	71.6	69.0	90.3	78.5	73.6
	W4A8	RTN	54.8	76.6	64.5	64.7	89.0	74.5	70.7
		SQ	51.6	76.6	64.2	65.7	89.1	74.4	70.3
		MBQ	55.6	76.5	64.4	65.7	89.2	73.3	70.8
InternVL2-26B	FP16	-	47.1	76.8	77.9	66.2	97.5	82.1	74.6
	W3A16	RTN	46.6	75.7	75.9	64.7	96.4	80.6	73.3
		GPTQ	44.8	75.8	76.0	60.9	96.3	80.1	72.3
		AWQ	46.4	76.2	76.4	64.5	96.7	81.0	73.5
		MBQ	47.1	76.3	76.5	64.5	97.3	81.1	73.8
	W4A8	RTN	44.7	76.0	76.4	62.6	96.7	79.6	72.7
		SQ	38.2	70.6	68.5	56.7	86.3	72.6	65.5
		MBQ	44.0	75.7	77.5	62.0	97.1	80.0	72.7
Qwen2-VL-72B	FP16	-	61.1	77.6	79.9	76.0	91.6	82.5	78.1
	W3A16	RTN	57.7	77.5	70.4	74.8	89.7	79.7	75.0
		GPTQ	57.3	77.2	78.5	73.6	91.5	81.6	76.6
		AWQ	59.6	77.6	79.6	75.4	90.4	82.4	77.5
		MBQ	59.6	77.7	79.4	75.6	90.5	82.5	77.6
	W4A8	RTN	58.1	76.6	66.2	71.3	90.1	77.0	73.2
		SQ	55.9	76.4	65.5	69.7	88.8	76.9	72.2
		MBQ	57.7	76.3	77.5	73.6	89.6	80.5	75.8

Table 3. Main results on the large models of LLaVA-onevision, InternVL2, and Qwen2-VL families. “SQ” is short for SmoothQuant.